
Table of Contents

1. What is SYLESS?
2. System Architecture
3. Frontend — React + Vite
4. Backend — Node.js + Express
5. The Compiler — How SYLESS Becomes Python
6. Phase 1: Lexer (Tokenizer)
7. Phase 2: Parser → AST
8. Phase 3: Code Generator → Python
9. Python Execution — Sandbox & Security
10. AI Tutoring — ARIA
11. DSA Visualization
12. What Makes SYLESS Special
13. Full Tech Stack Summary

SECTION 01

What is SYLESS?

 Save as PDF

SYLESS is a full-stack, AI-powered, browser-based programming IDE that lets users write code in **plain English sentences**. Instead of learning complex syntax like Python or JavaScript, beginners write naturally worded statements that are compiled into real Python code and executed live inside the browser.

Core Idea: Write English → SYLESS compiler converts it to Python → Python runs on the server → Output appears in the browser terminal — all in under a second.

A Simple Example

SYLESS CODE (WHAT THE USER WRITES)

```
make name = "Alice"
make age = 25
say "Hello {name}, you are {age}!"
check age ≥ 18 {
  say "You are an adult"
}
otherwise {
  say "You are a minor"
}
```

GENERATED PYTHON (AUTO-COMPILED)

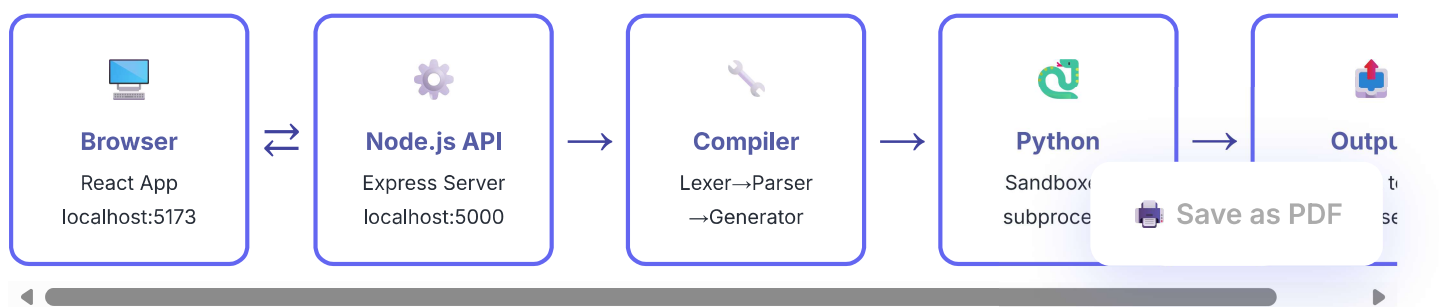
```
name = "Alice"
age = 25
print(f"Hello {name}, you are {age}!")
if (age ≥ 18):
    print("You are an adult")
else:
    print("You are a minor")
```

The user never sees the Python. They just see their output. The entire compilation happens transparently.

SECTION 02

System Architecture

SYLESS is split into two main applications that communicate over HTTP and WebSocket:





Frontend

React 18 + Vite
Tailwind CSS, Monaco Editor,
Framer Motion



Backend

Node.js + Express
REST API + WebSocket (ws library)



Database

MySQL via Sequelize
ORM with auto-sync and validation



AI Service

Groq + Gemini
Llama 3.3-70B + Gemini 2.0 Flash
(free APIs)

SECTION 03

Frontend — React + Vite

The frontend is a **React 18 single-page application** bundled with Vite. It runs entirely in the browser at `http://localhost:5173` and communicates with the backend via REST API calls.

Key Technologies

TECHNOLOGY	PURPOSE	WHY CHOSEN
React 18	UI framework	Component-based, fast re-rendering
Vite	Build tool & dev server	10x faster than Webpack, instant HMR
Tailwind CSS	Styling	Utility-first, no CSS files needed
Monaco Editor	Code editor in browser	Same engine as VS Code
Framer Motion	Animations	Smooth transitions and panel animations
Zustand	State management	Lightweight alternative to Redux
React Hot Toast	Notifications	Non-intrusive toast messages



Save as PDF

TECHNOLOGY	PURPOSE	WHY CHOSEN
Lucide React	Icons	Consistent SVG icon set

Key Pages & Components

- **IDE Editor** — Monaco-based code editor with SYLESS syntax, Run button, output terminal
- **AI Tutor (ARIA)** — Chat interface with course lessons, DSA topic explainers
- **DSA Visualizer** — Stack, Queue, Binary Tree, Sort visualizations (SVG/Canvas-based)
- **Auth Pages** — Login / Register with JWT-based session management
- **Community** — Share code snippets, like/comment posts
- **Learn Modules** — Structured curriculum with progress tracking

How the Editor Sends Code to the Server

```
// User clicks "Run" button in the IDE
const response = await fetch('/api/compile/run', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    code: editorContent, // raw SYLESS source code
    stdin: userInput      // optional input for ask → prompts
  })
});
const { output, error, executionTime } = await response.json();
```

JAVASCRIPT (FRONTEND)

SECTION 04

Backend — Node.js + Express

The backend is a **Node.js REST API** built with Express. It handles authentication, code compilation, AI requests, database operations, and real-time WebSocket collaboration.

API Routes

 Save as PDF

ROUTE	METHOD	WHAT IT DOES
/api/compile/run	POST	Compile SYLESS → Python → execute → return output
/api/compile/check	POST	Compile only (no execution) — syntax check
/api/ai/explain	POST	Send code to AI for explanation
/api/ai/debug	POST	Send error + code to AI for debugging help
/api/ai/chat	POST	General ARIA AI chat conversation
/api/auth/register	POST	Create new user account
/api/auth/login	POST	Login, returns JWT token
/api/dsa/*	GET/POST	DSA problems, solutions, visualization data
/api/learn/*	GET	Course modules, lessons, user progress
/api/community/*	GET/POST	Shared snippets, likes, comments
/ws	WebSocket	Live collaboration — broadcast code changes to room members

Security & Performance Middleware

- **Helmet.js** — Sets HTTP security headers (prevents XSS, clickjacking, etc.)
- **CORS** — Only allows requests from whitelisted frontend origins
- **Rate Limiting** — 30 compile requests/min, 20 AI requests/min per IP
- **Compression** — Gzip responses for faster data transfer
- **Morgan** — HTTP request logging for monitoring
- **JWT Auth** — JSON Web Tokens protect all authenticated routes

SECTION 05 — CORE INNOVATION

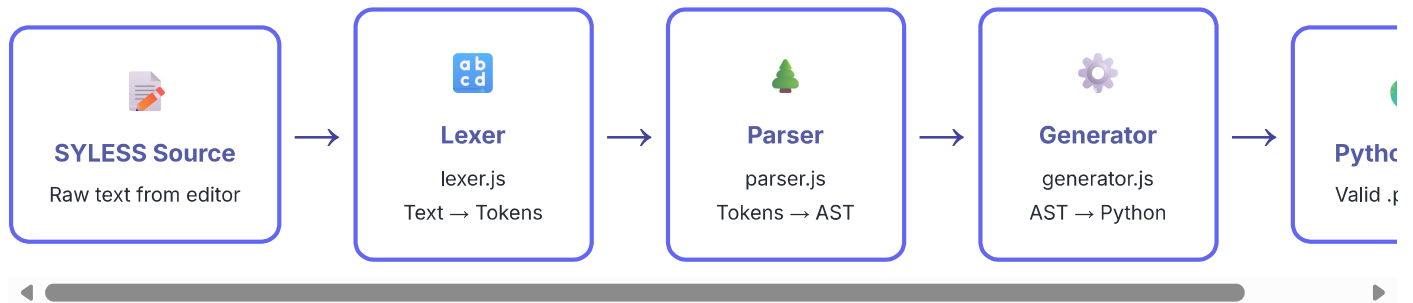
The Compiler — How SYLESS Becomes Python

 Save as PDF

This is the **heart of SYLESS**. A custom-built, three-phase compiler written in pure Node.js that transforms English-like SYLESS code into valid, executable Python 3 code.

The compiler pipeline has three phases, each a separate module:

Lexer (tokenizer) → Parser (AST builder) → Generator (Python code emitter)



Each phase can independently throw a friendly, user-readable error message pointing to the exact line number where the problem occurred — just like a professional IDE.

SECTION 06

Phase 1: Lexer — Turning Text into Tokens

The **Lexer** (also called a tokenizer or scanner) reads the raw SYLESS source code character-by-character and groups characters into meaningful units called **tokens**.

Analogy: Like reading a sentence word-by-word. The lexer splits `say "hello"` into: `[SAY]` `["hello"]`. It doesn't understand grammar yet — it just identifies the individual "words" of the language.

Token Types Recognized

KEYWORDS (MAP TO LANGUAGE CONSTRUCTS)

SYLESS WORD

`say / show`

TOKEN TYPE

`SAY`

OTHER TOKEN CATEGORIES

- **Literals:** NUMBER, STRING, BOOLEAN

- **Operators:** `+`, `-`, `*`, `/`, `%`, `^`

`>=`

 Save as PDF

SYLESS WORD

TOKEN TYPE

make

MAKE

ask

ASK

check

CHECK

otherwise

OTHERWISE

loop

LOOP

task

TASK

give

GIVE

push / pop /
insert

DSA
operations

train / predict

ML operations

- **Assignment:** =, +=, -=, *=, /=
- **Delimiters:** { } () [] , . :
- **Arrow:** → (used in say → "text")
- **Identifiers:** variable/function names
- **Comments:** lines starting with #
- **Special:** NEWLINE, EOF

Lexer in Action

Input: make age = 25

INPUT → TOKENS

Output tokens:

[MAKE, "make"] [IDENTIFIER, "age"] [ASSIGN, "="] [NUMBER, 25] [EOF]

Input: check x > 5 { say "big" }

Output tokens:

[CHECK] [IDENTIFIER, "x"] [GT, ">"] [NUMBER, 5] [LBRACE] [SAY] [STRING, "big"] [RBRACE]

Smart Feature: The lexer detects template strings (strings containing {variable}) and marks them with isTemplate = true so the generator can emit Python f-strings.

 Save as PDF

Phase 2: Parser — Building the AST

The **Parser** takes the flat list of tokens from the Lexer and builds an **Abstract Syntax Tree (AST)** — a hierarchical tree structure that represents the logical meaning of the code.

Analogy: If the Lexer reads individual words, the Parser understands grammar. It knows that `check` must be followed by a condition and a block `{ }`. The AST is like a diagram of how the sentences are structured.

AST Node Types

Print say → expression	VarDecl make x = value	Input ask name → prompt	IfStmt check cond { }
ForLoop loop N times { }	WhileLoop repeat while { }	ForEach for each x in arr { }	FuncDef task name(params) { }
FuncCall name(args)	Return give value	BinaryOp left op right	DSAMake make stack/queue
DSAPush push val into stack	Sort sort arr ascending	MLTrain train knn on data	MLPredict predict using model

Visual Example: AST for `check x > 5 { say "big" }`

Program

IfStmt

condition: BinaryOp

op: ">"

left: Identifier { name: "x" }

right: Literal { value: 5 }

body:

Print

AST TREE

Save as PDF


```
└─ value: Literal { value: "big" }  
└─ elseBody: null
```

How also check Chains into elif

```
check score ≥ 90 { say "A" }  
also check score ≥ 80 { say "B" }  
otherwise { say "C" }
```

SYLESS

```
if (score ≥ 90):  
    print("A")  
elif (score ≥ 80):  
    print("B")  
else:  
    print("C")
```

PYTHON (GENERATED)

SECTION 08

Phase 3: Code Generator — AST to Python

The **Generator** walks the AST tree node-by-node and emits valid **Python 3** source code. It tracks indentation level so the generated code is always correctly formatted.

Complete SYLESS → Python Mapping

SYLESS SYNTAX

```
say "Hello {name}"
```

```
make x = 10
```

```
ask name → "Enter name: "
```

```
check x > 5 { }
```

```
otherwise { }
```

GENERATED PYTHON

```
print(f"Hello {name}")
```

```
x = 10
```

```
name = _syless_input("Enter name: ")
```

```
if (x > 5):
```

```
else:
```

 Save as PDF

SYLESS SYNTAX

```
also check y = 0 { }

loop 5 times { }

loop i from 1 to 10 { }

repeat while x > 0 { }

for each item in fruits { }

task add(a, b) { give a + b }

make mystack

push 10 into mystack

pop from mystack

sort nums ascending

binary search 5 in nums

length of name

upper of name

train knn on data with labels as myModel
```

GENERATED PYTHON

```
elif (y == 0):

for _i in range(5):

for i in range(1, 10 + 1):

while (x > 0):

for item in fruits:

def add(a, b):
    return (a + b)

mystack = [] # SYLESS Stack

mystack.append(10)

mystack.pop()

nums.sort(reverse=False)

_bs_result = _syless_binary_search(nums, 5)
if _bs_result >= 0: print( ... )

len(name)

(name).upper()

myModel = KNeighborsClassifier()
myModel.fit(data, labels)
```

Auto-injected Helper Code

The generator automatically prepends helper functions to the Python output when needed:

INPUT HELPER (WHEN ask IS USED)

```
def _syless_input(prompt=""):
    s = input(prompt)
    try: return int(s)
```

DSA HELPERS (WHEN TREE/SEARCH IS USED)

```
def _syless_bst_insert(
    # Full BST insert logic
    ...
```



Save as PDF

```
except ValueError: pass
try: return float(s)
except ValueError: return s
```

Automatically converts typed input to int or float if possible — so users don't need to think about types.

```
def _syless_binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo ≤ hi:
        mid = (lo + hi) // 2
        ...
```

Python Execution — Sandboxed & Secure

After code generation, the Python source code is executed securely on the server. This is one of the most technically critical parts of SYLESS — running user-submitted code safely without exposing the server.

Security Risk: If a user writes `import os; os.system("rm -rf /")` and it runs on the server — catastrophic. SYLESS has multiple layers to prevent this.

Security Layer 1 — Static Code Analysis

Before execution, the generated Python is scanned with regex patterns that block dangerous operations:

```
// Automatically rejected before Python even starts:
BLOCKED_PATTERNS = [
    /import\s+os/,           // no filesystem access
    /import\s+subprocess/,   // no spawning child processes
    /import\s+sys/,          // no system manipulation
    /open\s*\(/,             // no file reading/writing
    /exec\s*\(/,             // no dynamic code execution
    /eval\s*\(/,             // no eval
    /globals\s*\(\s*\)/,     // no globals() introspection
    /getattr\s*\(/,          // no dynamic attribute access
]
```

BLOCKED PATTERNS (EXECUTOR.JS)

Security Layer 2 — Restricted Builtins Sandbox

A Python "sandbox prefix" is injected before every program that restricts which built-in functions are available:

```
# Only these safe built-ins are exposed to user code:
_safe_builtins = {
    'print': print, 'range': range, 'len': len,
    'int': int, 'float': float, 'str': str,
    'list': list, 'dict': dict, 'set': set,
    'abs': abs, 'max': max, 'min': min,
    'sorted': sorted, 'enumerate': enumerate,
    # ... ~25 total safe functions. No os, no sys, no file I/O.
}
```

SANDBOX PREFIX (PREPENDED TO EVERY EXECUTION)

Security Layer 3 — Process Isolation

The code runs in a **completely separate child process** (not in the main Node.js server process), with strict resource limits:



Timeout

10 seconds max

Infinite loops are killed automatically



Output limit

100 KB max output

Prevents output flooding attacks



Temp file

Random hex filename

Unique per run, deleted immediately after



Environment

Stripped env vars

Only PATH and Python encoding flags passed

User-Friendly Error Translation

Python errors are translated into beginner-friendly messages before being shown to the user:

PYTHON ERROR

`NameError: name 'x' is not defined`

`ZeroDivisionError`

`IndexError: list index out of range`

`RecursionError`

`Timeout (SIGTERM)`

SYLESS MESSAGE

'x' is not defined — did you use 'make' to create it first?

You tried to divide by zero — check your math!

You tried to access an item that does not exist in the array

Your recursive function went too deep — check the base case in your task

Your program took too long (10s limit). Check for infinite loops!

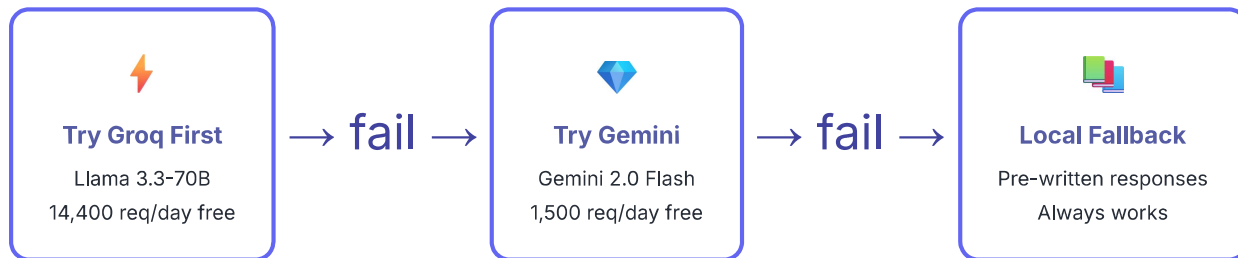


Save as PDF

AI Tutoring — ARIA

ARIA (AI Reasoning & Instruction Assistant) is the built-in AI tutor that explains code, debugs errors, and teaches concepts — all in the context of the SYLESS language.

AI Provider Chain (Free APIs, No Credit Card)



SYSTEM PROMPT

You are ARIA (AI Reasoning & Instruction Assistant), the built-in AI tutor for SYLESS — a beginner-friendly programming IDE where users write code in simple English-like sentences.

SYLESS Quick Reference:

- say → "text" → print text
- make x = 10 → variable declaration
- check x > 5 { } → if statement
- task name(param) { } → function definition

Your personality: encouraging, patient, simple analogies, avoid jargon. Always explain WHY. When showing code, always use SYLESS syntax first, then Python equivalent.

AI Features

- **Explain Code** — Sends the current editor code to AI for a step-by-step breakdown
- **Debug Error** — Sends the error message + code for targeted debugging help
- **Chat** — General programming conversation, always in SYLESS context
- **DSA Explain** — Ask ARIA to explain Stack, Queue, Binary Tree, Graph, etc.
- **Course Lessons** — Built-in curriculum with code examples that run in the IDE

 Save as PDF

DSA Visualization

SYLESS supports **Data Structures and Algorithms** directly in the language. Users can create stacks, queues, linked lists, binary trees, and graphs using natural English commands — and the IDE includes animated visual representations.

SYLESS DSA CODE

```
# Stack operations
make mystack
push 10 into mystack
push 20 into mystack
push 30 into mystack
pop from mystack
say mystack

# Binary search
make nums = [1, 3, 5, 7, 9]
sort nums ascending
binary search 7 in nums
```

GENERATED PYTHON

```
# Stack operations
mystack = []
mystack.append(10)
mystack.append(20)
mystack.append(30)
mystack.pop()
print(mystack)

# Binary search
nums = [1, 3, 5, 7, 9]
nums.sort(reverse=False)
_bs_result = _syless_binary_search(nums, 7)
if _bs_result ≥ 0:
    print("Found 7 at index", _bs_result)
```

Supported Data Structures

STRUCTURE	SYLESS MAKE COMMAND	PYTHON IMPLEMENTATION	OPERATIONS
Stack	make mystack	Python list	push, pop, say
Queue	make myqueue	Python collections.deque	insert, remove, say
Linked List	make mylinkedlist	Python list	add, say
Binary Tree (BST)	make mytree	Custom _Node class	insert, search
Graph	connect A to B	Python dict of adjacency lists	

What Makes SYLESS Special



Custom Compiler

Built from scratch

No parser generators (ANTLR, PEG) used — hand-written recursive descent parser in Node.js



English Syntax

Natural language coding

No semicolons, no colons, no indentation rules — just say, make, check, loop



DSA in English

Data structures built-in

Stack, Queue, Tree, Graph with natural English commands — no OOP needed



Real-time

WebSocket collaboration

Multiple users can code in the same room — changes broadcast live



Sandboxed Execution

Multi-layer security

Static analysis + restricted builtins + process isolation + 10s timeout



AI Integration

Groq + Gemini fallback

Always works — 3-tier fallback chain ensures AI is always available



ML Support

scikit-learn in English

train knn on data — generates real sklearn code for KNN, SVM, Random Forest, etc.



Beginner-First

Errors in plain English

Python errors translated into human-readable messages with hints

The Big Technical Achievement

Most "learn to code" platforms wrap existing languages with tutorials. **SYLESS built an entirely new programming language from scratch** — with its own tokenizer, grammar, generator — that compiles to Python. This is the same fundamental architecture

 Save as PDF

languages like TypeScript (compiles to JS), Kotlin (compiles to JVM bytecode), and CoffeeScript (compiles to JS).

The difference is SYLESS chose **Python as the compilation target** because Python has the richest ecosystem of libraries (NumPy, scikit-learn, etc.), making SYLESS programs capable of serious computation despite looking like English.

SECTION 13

Full Technology Stack Summary

LAYER	TECHNOLOGY	VERSION/DETAILS	ROLE
Frontend	React	18	UI framework
	Vite	5.x	Build tool & HMR dev server
	Tailwind CSS	3.x	Utility-first styling
	Monaco Editor	latest	VS Code engine in browser
	Framer Motion	latest	UI animations
	Zustand	latest	Global state management
Backend	Node.js	18+	Server runtime
	Express.js	4.x	HTTP API framework
	ws (WebSocket)	latest	Real-time collaboration
	Sequelize ORM	6.x	MySQL database access
	Helmet.js	latest	HTTP security headers
	express-rate-limit	latest	API rate limiting
Compiler	jsonwebtoken	latest	JWT authentication
	lexer.js	Custom	Tokenizer (text → tokens)



Save as PDF

LAYER	TECHNOLOGY	VERSION/DETAILS	ROLE
Execution	parser.js	Custom	AST builder (tokens → tree)
	generator.js	Custom	Code emitter (tree → Python)
	Python 3	3.8+	Runtime for SYLESS programs
	child_process	Node built-in	Sandboxed subprocess execution
AI	Groq API	Llama 3.3-70B	Primary AI (free, 14,400 req/day)
	Gemini API	Gemini 2.0 Flash	Fallback AI (free, 1,500 req/day)
Database	MySQL	via Sequelize	Users, progress, community posts

How a Single Run Request Flows Through the Entire System

- User types SYLESS code in the Monaco editor and clicks "Run"**

The React frontend sends a POST request to `/api/compile/run` with the code as JSON.
- Express rate limiter checks the IP**

Maximum 30 compile requests per minute. If exceeded, returns a friendly error.
- Compiler Phase 1: Lexer scans the source**

Converts raw text into a flat list of typed tokens (SAY, IDENTIFIER, NUMBER, etc.).
- Compiler Phase 2: Parser builds the AST**

Groups tokens into a tree of nodes (IfStmt, FuncDef, Print, etc.) following SYLESS grammar rules.
- Compiler Phase 3: Generator emits Python**

Walks the AST and produces valid Python 3 source code with correct indentation.
- Security check on generated Python**

Regex patterns scan for `import os` , `eval()` , `open()` , etc. Blocked if detected.
- Python written to a temp file and executed as a child process**

The sandbox prefix is prepended. A 10-second timeout is applied. Output is capped at 100 KB.
- Temp file deleted immediately after execution**

The Node.js backend reads stdout/stderr from the completed process.
- Python errors translated to user-friendly messages**

`NameError` → "did you use 'make' to create it?", `ZeroDivisionError` → "check your math!", etc.



Save as PDF

10 JSON response returned to frontend

`{ output, error, executionTime, pythonCode, tokens, ast }` — displayed in the terminal panel.

SYLESS — AI-Powered English-Like Programming IDE

Built with React · Node.js · Custom Compiler · Python · Groq AI

Technical Document · May 2026

 Save as PDF